

Documentacion Tecnica Viva

August 6, 2026

CONTENTS

Documentacion Tecnica Viva	
1. Proposito del documento	
2. Estrategia recomendada de documentacion	
3. Skills y mecanismos adecuados detectados	
3.1 document-generate	
3.2 document-release	
3.3 make-pdf	
4. Estado actual del proyecto	
4.1 Tipo de solucion	
4.2 Modulos principales	
5. Cambios tecnicos recientes documentados	
5.1 Flujo de staging y promocion controlada	
5.2 Fortalecimiento de configuracion y fallback seguro	
5.3 Mejora profesional de filtros y experiencia de uso en la matriz	
5.4 Correccion de bloqueo de interaccion en la interfaz	
5.5 Wrappers backend para cache, roles y ayudas de busqueda	
6. Agentes, herramientas y skills utilizados o definidos para este proceso	
6.1 Agente principal de implementacion	
6.2 Skills de gstack relevantes para documentacion y evidencia	
6.3 Skills de gstack relevantes para mejora de interfaz	
6.4 Herramientas de verificacion ya usadas en este proyecto	
6.5 Segundo agente de implementacion (sesion paralela)	
7. Politica de actualizacion obligatoria de este documento	
8. Flujo de trabajo documental recomendado para futuras modificaciones	
8.1 Antes de modificar un archivo	
8.2 Durante la modificacion	
8.3 Despues de la modificacion	
9. Plantilla para futuras entradas	

9.1 Plantilla vigente (actualizada 2026-08-05)	
9.2 Plantilla original (version inicial, Copilot)	
10. Exportacion a PDF o Word	
10.1 Situacion actual	
10.2 Camino recomendado para PDF	
10.3 Camino recomendado para Word	
11. Decision adoptada	
12. Registro de intervenciones — Agente Claude Code (Claude Sonnet 5, orquestado con gstack)	
12.1 [2026-08-05] [SEC-P0] Autorizacion server-side en registro de seguimiento	
12.2 [2026-08-05] [SEC-P1] Sanitizacion de XSS almacenado (OBSERVACIONES / SITUACIONES)	
12.3 [2026-08-05] [CONC-P2] Concurrencia — LockService en motores de reglas y modulo PAC	
12.4 [2026-08-05] [NS-01] Resolucion de colision de namespace onOpen()	
12.5 [2026-08-05] [FE-01] Desacoplamiento de Index.html (CSS y logica JS principal)	
12.6 Correccion sobre disponibilidad de make-pdf (ver Secciones 3.3 y 10)	
12.7 [2026-08-05] [CONC-P2.1] Diferenciar timeout de lock vs JSON invalido en motores de reglas	
12.8 [2026-08-05] [PLAN-FE-02] Dictamen de arquitectura — subdivisión modular de app_js.html	
12.9 [2026-08-05] [CONC-FE-02] Subdivisión modular de app_js.html — Fases 1 y 2 (Build)	
12.10 [2026-08-06] [CONC-FE-02] Subdivisión modular de app_js.html — Fase 3 (Build) — cierre acumulado Fases 1-3	
13. Inventario Exhaustivo del Repositorio	
13.1 Nucleo del backend	
13.2 Frontend (Index.html y los dos archivos extraidos en FE-01)	
13.3 Subproyecto MatrizSeguimiento_script/ (independiente del backend principal)	
13.4 Subproyecto normalizacion_script/ (independiente del backend principal)	
14. Matriz Consolidada de Metricas deCodigo	
15. Mapeo del Ciclo de Vida por Especialistas gstack (Think -> Reflect)	
16. Manual de Gobierno Multi-Agente	
16.1 Agentes activos	
16.2 Reglas de convivencia	
16.3 Verificacion cruzada de hallazgos	
17. Guia de Exportacion e Integracion Continuada (actualizada 2026-08-05)	

17.1 Exportacion directa a Word (via gstack make-pdf CLI)

17.2 Exportacion a PDF con portada y tabla de contenidos

17.3 Estado verificado de estos comandos (2026-08-05, verificado dos veces el mismo dia)

17.4 Ruta alternativa (Pandoc)

Documentacion Tecnica Viva

1. Proposito del documento

Este documento es la fuente oficial de evidencia tecnica del proyecto APLICACION DE PREDIOS. Su objetivo es registrar, en espanol y con lenguaje tecnico facil de entender, la arquitectura del aplicativo, los cambios funcionales realizados, los archivos intervenidos, las validaciones ejecutadas y los agentes o skills utilizados durante el proceso.

Este archivo debe actualizarse cada vez que se modifique un modulo relevante, se agregue una funcion, se cambie el comportamiento de la interfaz o se altere la configuracion del sistema.

2. Estrategia recomendada de documentacion

La estrategia mas robusta no es escribir directamente en Word. La estrategia recomendada es esta:

1. Mantener este documento en Markdown como fuente canonica versionable.
2. Actualizarlo junto con cada cambio de codigo.
3. Exportarlo a PDF o DOCX cuando se necesite una evidencia formal para entrega.

Ventajas de esta estrategia:

- Permite control de cambios junto al codigo.
- Facilita enlazar archivos, funciones y puntos modificados.
- Evita perder trazabilidad cuando el contenido crece.
- Permite generar salidas finales para auditoria, comites o supervisores.

3. Skills y mecanismos adecuados detectados

Durante la investigacion se confirmo que si existen skills adecuadas para sostener este proceso documental:

3.1 DOCUMENT-GENERATE

Uso recomendado: crear documentacion base de un proyecto, modulo, feature o flujo completo.

Valor para este proyecto:

- Sirve para producir la primera version de la documentacion tecnica.
- Encaja bien para describir modulos como Codigo.js, config.js, Index.html y PAC.
- Es apropiado para generar explicaciones estructuradas con enfoque tecnico.

3.2 DOCUMENT-RELEASE

Uso recomendado: actualizar la documentacion despues de cambios o entregas.

Valor para este proyecto:

- Sirve para mantener alineados los documentos con el codigo que va cambiando.

- Es el skill mas cercano a una bitacora de mantenimiento posterior.
- Debe usarse cuando ya hubo modificaciones implementadas y validadas.

3.3 MAKE-PDF

Uso recomendado: convertir un Markdown en un PDF formal.

Estado actual del entorno:

- El binario de gstack para PDF no esta disponible en este equipo en este momento.
- No se detecto Pandoc instalado para generar DOCX o PDF automaticamente.

Conclusion practica:

- Si hoy se requiere documentar, se puede hacer de inmediato en Markdown.
- Cuando se necesite entregable formal, se recomienda habilitar uno de estos caminos:
 - Instalar Pandoc para generar `.docx` y `.pdf`.
 - Construir el binario de `/make-pdf` desde gstack para exportar a PDF.

4. Estado actual del proyecto

4.1 TIPO DE SOLUCION

Aplicacion web en Google Apps Script con interfaz HTML renderizada por `HtmlService`, integracion con Google Sheets como fuente principal de datos, automatizaciones con triggers y modulos especializados para PAC, auditoria, permisos y reportes.

4.2 MODULOS PRINCIPALES

- `[Codigo.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Codigo.js)`: orquestacion backend, menu de Google Sheets, carga principal del tablero y funciones publicas.
- `[config.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\config.js)`: configuracion central del sistema y resolucion de IDs de archivos.
- `[Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html)`: interfaz principal del tablero, filtros, matriz y experiencia del usuario.
- `[cache_backend.gs](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\cache_backend.gs)`: wrappers de backend para cache, roles y ayudas de busqueda.
- `[pac_api.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\pac_api.js)`: logica del modulo PAC.

5. Cambios tecnicos recientes documentados

5.1 FLUJO DE STAGING Y PROMOCION CONTROLADA

Objetivo funcional

Permitir validar un archivo de staging (Dato 2) antes de promoverlo al archivo principal (Dato 1), reduciendo el riesgo de sobrescribir informacion productiva sin control.

Archivo intervenido

- [Codigo.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Codigo.js#L1845)

Cambios realizados

- Se amplio el menu de control en Google Sheets para incluir acciones de validacion y promocion de staging.
- Se agrego una validacion estructural entre hojas de staging y produccion.
- Se incorpore un flujo de promocion con confirmacion de usuario y `LockService`.
- Se agrego copia controlada por hoja entre archivo staging y archivo principal.

Referencias puntuales

- Menu de control: [Codigo.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Codigo.js#L1845)
- Entrada de validacion: [Codigo.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Codigo.js#L1849)
- Funcion de validacion: [Codigo.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Codigo.js#L1873)
- Funcion de promocion: [Codigo.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Codigo.js#L1923)
- Copia de hojas: [Codigo.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Codigo.js#L1962)

Impacto tecnico

- Se formalizo un ciclo de preproduccion y promocion manual.
- Se redujo la dependencia de cambios directos sobre el archivo principal.
- Se deajo base para gobierno de datos y validacion previa al despliegue funcional.

5.2 FORTALECIMIENTO DE CONFIGURACION Y FALLBACK SEGURO**Objetivo funcional**

Eliminar dependencias rigidas de IDs hardcodeados y permitir que el sistema opere con mayor seguridad en entornos de pruebas, staging o despliegue local.

Archivo intervenido

- [config.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\config.js#L16)

Cambios realizados

- Se incorpore `DATA_FILES.STAGING` como configuracion de archivo secundario para validacion y promocion.
- Se mantuvo compatibilidad con `DATA_FILES_IDS`.
- Se reforzo la resolucion de IDs de archivos usando rutas alternativas.
- Se agrego fallback al spreadsheet activo cuando `DATA_FILES.PRINCIPAL` no esta configurado.

Referencias puntuales

- Estructura de archivos de datos: [config.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\config.js#L16)
- Compatibilidad `DATA_FILES_IDS`: [config.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\config.js#L23)
- Resolucion de lista de archivos: [config.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\config.js#L270)
- Validacion central de configuracion: [config.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\config.js#L294)
- Obtencion de ID de staging: [config.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\config.js#L678)
- Resolucion global de IDs: [config.js](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\config.js#L707)

Impacto tecnico

- El sistema queda mas portable entre ambientes.
- Se reduce el riesgo de fallar por configuracion incompleta.
- Se mejora la mantenibilidad del backend.

5.3 MEJORA PROFESIONAL DE FILTROS Y EXPERIENCIA DE USO EN LA MATRIZ

Objetivo funcional

Mejorar la experiencia del usuario en el tablero principal permitiendo filtros mas rapidos, mas precisos y con menor friccion operacional.

Archivo intervenido

- [Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html#L202)

Cambios realizados

- Se agregaron campos de busqueda para Proyecto, Tramo y Estado.
- Se habilito filtrado incremental mientras el usuario escribe.
- Se incorporaron indicadores de coincidencias para orientar la busqueda.
- Se mejoro el comportamiento de limpieza y reseteo de filtros dependientes.
- Se reforzo la usabilidad con interacciones de teclado como Enter, Escape y flecha abajo.

Referencias puntuales

- Busqueda de proyecto: [Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html#L202)
- Meta informativa de proyecto: [Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html#L203)
- Busqueda de tramo: [Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html#L211)
- Meta informativa de tramo: [Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html#L212)

- Búsqueda de estado: [Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html#L220)
- Meta informativa de estado: [Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html#L221)

Impacto tecnico

- La interfaz es mas util para bases grandes.
- Se reduce el tiempo de localizacion de proyectos y estados.
- Se deja una base escalable para futuras mejoras de UX.

5.4 CORRECCION DE BLOQUEO DE INTERACCION EN LA INTERFAZ

Objetivo funcional

Resolver un bloqueo visual que impedia seleccionar elementos dentro del HTML cargado por Apps Script.

Archivo intervenido

- [Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html#L832)

Cambios realizados

- Se detecto un conflicto entre clases globales `.modal` y `.modal-content` usadas por Bootstrap y un modal custom de la seccion Filtro Matriz.
- Se aislaron los estilos del modal custom bajo clases propias para evitar que sobrescribieran el resto de la aplicacion.

Referencias puntuales

- Modal corregido: [Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html#L832)
- Contenedor del modal: [Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html#L833)
- Estilo aislado del overlay: [Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html#L861)
- Estilo aislado del contenido: [Index.html](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\Index.html#L862)

Impacto tecnico

- Se recupero la interaccion normal del tablero.
- Se evito una regresion transversal sobre todos los modales Bootstrap.

5.5 WRAPPERS BACKEND PARA CACHE, ROLES Y AYUDAS DE BUSQUEDA

Objetivo funcional

Crear una capa de integracion mas segura entre el frontend y Apps Script para roles, invalidacion de cache y sugerencias de busqueda.

Archivo intervenido

- [cache_backend.gs](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\cache_backend.gs#L23)

Referencias puntuales

- Roles de usuario: [cache_backend.gs](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\cache_backend.gs#L23)
- Consulta de operacion de cache: [cache_backend.gs](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\cache_backend.gs#L57)
- Invalidacion de cache: [cache_backend.gs](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\cache_backend.gs#L90)
- Sugerencias de busqueda: [cache_backend.gs](e:\PROYECTOS\CLAUDE CODE\CREACIÓN APK\Aplicación de Predios\cache_backend.gs#L251)

6. Agentes, herramientas y skills utilizados o definidos para este proceso

6.1 AGENTE PRINCIPAL DE IMPLEMENTACION

- GitHub Copilot sobre GPT-5.4.

6.2 SKILLS DE GSTACK RELEVANTES PARA DOCUMENTACION Y EVIDENCIA

- `document-generate` : para redactar documentacion base.
- `document-release` : para actualizar documentacion despues de cambios.
- `make-pdf` : para generar PDF formal desde Markdown cuando el binario este disponible.

6.3 SKILLS DE GSTACK RELEVANTES PARA MEJORA DE INTERFAZ

- `design-review` : auditoria visual y de experiencia de usuario.
- `design-html` : refinamiento e implementacion de HTML/CSS.
- `design-consultation` : definicion de lineamientos de disenio.

6.4 HERRAMIENTAS DE VERIFICACION YA USADAS EN ESTE PROYECTO

- `clasp push` para publicar cambios en Apps Script.
- Validacion de errores de sintaxis en archivos clave.
- Lectura puntual de codigo y verificaciones de lineas afectadas.

6.5 SEGUNDO AGENTE DE IMPLEMENTACION (SESION PARALELA)

- **Claude Code (modelo Claude Sonnet 5), orquestado con el conjunto de skills gstack.**
- Trabaja sobre el mismo repositorio y la misma rama de git (`fix/qa-saveTracking-batch-lock`) que la sesion de GitHub Copilot descrita en 6.1, en momentos distintos.

- Pipeline tipico usado en esta sesion: `/plan-eng-review` (diagnostico arquitectonico) -> `/review` (auditoria Staff Engineer) -> implementacion directa de parches -> `/review` de verificacion -> `/careful` (guardarrailes) -> `clasp push` -> `/qa` + `/setup-browser-cookies` (QA en navegador) -> `/context-save` (continuidad entre sesiones) -> `/sync-gbrain` (memoria persistente local).
- Todas las intervenciones de esta sesion quedan registradas en la Seccion 12, siguiendo la misma plantilla de la Seccion 9.

7. Politica de actualizacion obligatoria de este documento

Cada vez que se modifique un archivo relevante del proyecto, se debe actualizar este documento con minimo los siguientes puntos:

1. Nombre del archivo intervenido.
2. Proposito del archivo dentro de la solucion.
3. Motivo del cambio.
4. Resumen funcional de lo modificado.
5. Referencia puntual a funciones, bloques o lineas afectadas.
6. Validacion ejecutada despues del cambio.
7. Estado del despliegue o publicacion.

8. Flujo de trabajo documental recomendado para futuras modificaciones

8.1 ANTES DE MODIFICAR UN ARCHIVO

Registrar en este documento:

- Que hace el archivo.
- Que problema o mejora se va a abordar.
- Que riesgo funcional existe si se modifica.

8.2 DURANTE LA MODIFICACION

Registrar:

- Funciones nuevas o actualizadas.
- Secciones de UI, backend o configuracion afectadas.
- Decisiones tecnicas adoptadas.

8.3 DESPUES DE LA MODIFICACION

Registrar:

- Validaciones ejecutadas.
- Resultado de compilacion, despliegue o push.
- Impacto esperado en usuario, operacion o mantenimiento.

9. Plantilla para futuras entradas

9.1 PLANTILLA VIGENTE (ACTUALIZADA 2026-08-05)

Esta es la plantilla a usar para toda intervencion nueva a partir del 2026-08-05. Amplia la plantilla original (ver 9.2) con identificador de tarea, metricas de LOC y checklist de validacion, para que el documento funcione tambien como evidencia auditable, no solo como narrativa.

```
### [AAAA-MM-DD] [Identificador de la Tarea/PR] - Nombre Corto del Cambio

**Archivo(s) Intervenido(s):**
- `ruta/al/archivo.ext#L10-L45`

**Proposito del Archivo en el Sistema:**
[Descripcion breve del rol del archivo en la arquitectura]

**Motivo de la Intervencion:**
[Explicacion del problema, vulnerabilidad o requerimiento de negocio]

**Cambios Tecnicos Realizados:**
- Detalle 1
- Detalle 2

**Metricas del Cambio:**
- LOC Anadidas: X | LOC Eliminadas: Y | Variacion Neta: Z

**Validacion y Pruebas Ejecutadas:**
- [x] Sintaxis validada con `node --check`
- [x] Auditoria con `/review`
- [x] Pruebas en navegador con `/qa` sobre URL de desarrollo

**Impacto en Produccion:**
[Resumen del beneficio tecnico, operacional o de seguridad obtenido]
```

Notas de uso:

- **Identificador de la Tarea/PR:** este proyecto aun no tiene un sistema formal de tickets/PR (el repositorio no tiene remoto configurado). Mientras eso no exista, usar como identificador la misma etiqueta que se deja en el comentario del codigo (por ejemplo `SEC-P1`, `SEC-P1.5`) o un slug corto y consistente (`NS-01`, `FE-01`) si el cambio no lleva etiqueta en el codigo.
- **Metricas del Cambio:** reportar solo lineas atribuibles a la intervencion documentada, no el `git diff --numstat` crudo del archivo completo — un archivo puede acumular cambios de varias intervenciones o sesiones distintas en el mismo commit. Si el archivo es nuevo (no existia antes en el repositorio), aclararlo explicitamente en vez de reportar “LOC Eliminadas: 0” sin contexto.
- **Validacion y Pruebas Ejecutadas:** marcar solo lo que realmente se ejecuto en esa intervencion especifica; dejar sin marcar (`[ ]`) lo que no aplico, no borrarlo de la lista.

9.2 PLANTILLA ORIGINAL (VERSION INICIAL, COPILOT)

Se conserva para referencia historica; las entradas de la Seccion 5 fueron escritas con este formato.

```

### [Fecha] Nombre corto del cambio

**Archivo intervenido**

- [ruta/al/archivo](ruta/al/archivo#L1)

**Que hace el archivo**

Breve descripcion del rol del archivo dentro del sistema.

**Motivo del cambio**

Problema, mejora o ajuste solicitado.

**Cambios realizados**

- Punto 1
- Punto 2
- Punto 3

**Referencias puntuales**

- [ruta/al/archivo](ruta/al/archivo#L1)
- [ruta/al/archivo](ruta/al/archivo#L1)

**Validacion ejecutada**

- Comando o comprobacion realizada.
- Resultado.

**Impacto**

Resumen de beneficio tecnico y funcional.

```

10. Exportacion a PDF o Word

10.1 SITUACION ACTUAL

- Hoy ya puede mantenerse este documento en Markdown.
- Hoy no se detecto Pandoc instalado en el entorno.
- Hoy no se detecto listo el binario local de gstack para `make-pdf`.

10.2 CAMINO RECOMENDADO PARA PDF

Cuando se habilite `make-pdf` o Pandoc, este documento podra exportarse como entregable formal.

10.3 CAMINO RECOMENDADO PARA WORD

Si se requiere `.docx`, la recomendacion es instalar Pandoc y generar snapshots del documento vivo cuando se necesiten radicados, informes o anexos.

11. Decision adoptada

Se adopta este archivo como bitacora tecnica viva del proyecto. A partir de este punto, toda mejora relevante del sistema debe dejar evidencia aqui antes de cerrar la intervencion tecnica.

12. Registro de intervenciones — Agente Claude Code (Claude Sonnet 5, orquestado con gstack)

Nota de coautoría: este documento fue creado inicialmente por otra sesión de trabajo (GitHub Copilot sobre GPT-5.4, ver Sección 6.1). Las entradas de esta sección documentan una sesión distinta (Claude Code, ver Sección 6.5), ejecutada sobre la misma rama de git. Ambas sesiones comparten este archivo como bitacora única del proyecto, tal como establece la Sección 7.

12.1 [2026-08-05] [SEC-P0] AUTORIZACION SERVER-SIDE EN REGISTRO DE SEGUIMIENTO

Archivo(s) Intervenido(s):

- `Codigo.js#L51-L62`

Propósito del Archivo en el Sistema: `Codigo.js` es el orquestador principal del backend: contiene `doGet`, `saveFollowupData` (punto de entrada real para que el frontend registre seguimiento de un RT) y `saveTrackingData` (la lógica que escribe en la matriz principal).

Motivo de la Intervención: Una auditoría tipo Staff Engineer detectó que `saveFollowupData` derivaba la identidad del usuario vía `Session.getActiveUser()` pero nunca verificaba su rol contra `GestorPermisos` antes de escribir. Cualquier usuario autenticado del dominio — sin importar su rol — podía invocar la función desde la consola del navegador y registrar seguimiento; solo la interfaz ocultaba el botón, no había control de autorización real en el servidor. Severidad P0 (falla de control de acceso sobre una escritura de datos reales).

Cambios Técnicos Realizados:

- Se agregó validación obligatoria: `GestorPermisos.obtenerRol(userEmail)` debe ser `EDITOR` o `ADMIN`; si no, la función retorna error y no escribe nada.
- El bloqueo queda registrado con `console.warn` para trazabilidad.

Fragmento de Código (Antes vs Despues):

Antes (`Codigo.js` , dentro de `saveFollowupData`):

```
function saveFollowupData(dataJson) {
  try {
    const data = typeof dataJson === 'string' ? JSON.parse(dataJson) : dataJson;
    const userEmail = Session.getActiveUser().getEmail();
    const formObject = {
```

Despues:

```
function saveFollowupData(dataJson) {
  try {
    const userEmail = Session.getActiveUser().getEmail();

    // SEC-P0: validacion de autorizacion server-side (no confiar solo en la UI)
    const gestorPermisos = new GestorPermisos();
    const rolUsuario = gestorPermisos.obtenerRol(userEmail);
    const rolesAutorizados = [getConfig('ROLES.EDITOR'), getConfig('ROLES.ADMIN')];
    if (!rolUsuario || rolesAutorizados.indexOf(rolUsuario) === -1) {
      console.warn(`Acceso denegado a saveFollowupData para ${userEmail} (rol: ${rolUsuario} || 'ninguno')`);
      return { success: false, error: 'No tiene permisos para registrar seguimiento. Se requiere rol Editor o Administrador.' };
    }

    const data = typeof dataJson === 'string' ? JSON.parse(dataJson) : dataJson;
    const formObject = {
```

Metricas del Cambio:

Metrica	Valor
LOC iniciales (bloque afectado)	5
LOC finales (bloque afectado)	15
Delta absoluto	+10
% variacion (bloque)	+200%

Nota de alcance: esta tabla mide solo el bloque de codigo mostrado arriba, no el archivo completo. `Codigo.js` paso de 1863 a 1985 lineas totales entre los mismos dos commits (ver Seccion 14), pero ese delta de archivo completo incluye otras intervenciones ademas de esta.

Validacion y Pruebas Ejecutadas:

- ☒ Sintaxis validada con `node --check Codigo.js`
- ☒ Auditoria con `/review` (verifico compatibilidad de `GestorPermisos` con la llamada)
- ☐ Pruebas en navegador con `/qa` sobre URL de desarrollo (cubierto indirectamente en 12.5, no se repitio prueba aislada de este flujo)

Impacto en Produccion: Cierra una vulnerabilidad de control de acceso: la escritura de seguimiento ahora exige rol Editor o Administrador, verificado en el servidor, no solo en la interfaz.

12.2 [2026-08-05] [SEC-P1] SANITIZACION DE XSS ALMACENADO (OBSERVACIONES / SITUACIONES)

Archivo(s) Intervenido(s):

- `Index.html#L4103-L4111` (funcion nueva)
- `Index.html#L5469, L5650, L5740, L6288, L6297` (5 puntos de aplicacion)

Proposito del Archivo en el Sistema: Plantilla principal del tablero (`HtmlService`). Muestra la matriz de predios, el historial de seguimiento por RT y los formularios de edicion.

Motivo de la Intervencion: El campo `OBSERVACIONES` (texto libre capturado del usuario) se interpolaba sin escapar directamente en `innerHTML` / `insertAdjacentHTML` en 5 puntos distintos del archivo. Cualquier usuario con permiso de edicion podia inyectar HTML/JavaScript que se ejecutaria en el navegador de cualquier otro usuario (incluido un Administrador) al ver el historial de ese RT.

Cambios Tecnicos Realizados:

- Se creo una funcion reutilizable `escapeHtml()`.
- Se aplico en los 5 puntos de renderizado identificados: lista de cambios pendientes, banner de “predio inactivo” (dos vistas distintas), timeline de seguimiento y tabla generica de detalle de RT.

Fragmento deCodigo (Antes vs Despues):

Funcion nueva (`Index.html#L4103-L4111`):

```
function escapeHtml(text) {
  if (text === null || text === undefined) return '';
  return String(text)
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#39;');
}
```

Ejemplo de aplicacion en punto de uso — timeline de seguimiento (`Index.html#L5728` , hallazgo original de la auditoria):

Antes:

```
<small class="wrap-obs" style="max-width: 250px; display: block;">
  ${record.observaciones}
</small>
```

Despues:

```
<small class="wrap-obs" style="max-width: 250px; display: block;">
  ${escapeHtml(record.observaciones)}
</small>
```

Metricas del Cambio:

Metrica	Valor
LOC iniciales (funcion + 5 puntos de uso)	16 (5 lineas de codigo preexistente en los 5 puntos de uso)
LOC finales	27 (11 lineas de funcion nueva + 5 puntos de uso modificados)
Delta absoluto	+11
% variacion	+69%

Nota de alcance: cifra atribuible solo a esta intervencion, separada de la extraccion de 12.5 que toca el mismo archivo (`Index.html`) en la misma sesion.

Validacion y Pruebas Ejecutadas:

- ☒ Sintaxis validada con `node --check` (bloque `<script>` completo)
- ☒ Auditoria con `/review`
- ☒ Pruebas en navegador con `/qa` sobre URL de desarrollo (verificado junto con 12.5, mismo deployment)

Impacto en Produccion: Elimina un riesgo de robo de sesion o phishing interno entre usuarios del mismo dominio (incluye a Administradores) via el campo de observaciones del seguimiento.

12.3 [2026-08-05] [CONC-P2] CONCURRENCIA — LOCKSERVICE EN MOTORES DE REGLAS Y MODULO PAC

Archivo(s) Intervenido(s):

- `evaluador_alertas.js#L8-L75, L348-L417`
- `motor_reglas.js#L8-L75`
- `pac_gestor.js#L867-L919, L955-L1069`

Proposito del Archivo en el Sistema: `evaluador_alertas.js` / `motor_reglas.js` implementan el motor de reglas de negocio que calcula el semaforo de riesgo y genera alertas. `pac_gestor.js` gestiona la sincronizacion, el borrador y la aprobacion del modulo PAC (seguimiento presupuestal).

Motivo de la Intervencion: Los tres archivos escribian hojas compartidas (`CONFIG REGLAS` , `ALERTAS_ACTIVAS` , `PAC_Vigente` , `PAC_Borrador`) sin usar `LockService` , a diferencia del resto del sistema. Dos ejecuciones simultaneas (por ejemplo, dos administradores aprobando un borrador PAC al mismo tiempo, o el motor de alertas corriendo mientras alguien edita las reglas) podian generar perdida de escrituras o filas duplicadas.

Cambios Tecnicos Realizados:

- `LockService.getScriptLock()` con el patron `try { lock.waitLock(20000) ... } catch {} finally { lock.releaseLock() }` en: `obtenerReglasJSON` , `guardarReglasJSON` y `_guardarAlertasEnHoja` (ambos archivos de reglas); `aprobarBorradorPAC` y `_pac_compararYGenerarBorrador` (PAC).
- En `obtenerReglasJSON` se agrego ademas doble verificacion (comprobar de nuevo tras adquirir el lock) para evitar crear la hoja `CONFIG REGLAS` dos veces bajo concurrencia.

Fragmento deCodigo (Antes vs Despues):

Antes (`pac_gestor.js` , inicio de `aprobarBorradorPAC`):

```
function aprobarBorradorPAC(motivo) {
  pac_log('aprobarBorradorPAC: ' + (motivo || 'aprobacion manual'));
  try {
    const ss = pac_getSpreadsheet();
    ...
  } catch (e) {
    pac_log('aprobarBorradorPAC ERROR: ' + e.message, 'ERROR');
    return { success: false, mensaje: 'Error al aprobar borrador: ' + e.message };
  }
}
```

Despues:

```
function aprobarBorradorPAC(motivo) {
  pac_log('aprobarBorradorPAC: ' + (motivo || 'aprobacion manual'));
  // CONC-P2: LockService -- publica el borrador sobre PAC_Vigente (escritura masiva compartida)
  const lock = LockService.getScriptLock();
  try {
    lock.waitLock(20000);
    const ss = pac_getSpreadsheet();
    ...
  } catch (e) {
    pac_log('aprobarBorradorPAC ERROR: ' + e.message, 'ERROR');
    return { success: false, mensaje: 'Error al aprobar borrador: ' + e.message };
  } finally {
    try { lock.releaseLock(); } catch (er) {}
  }
}
```

Metricas del Cambio:

Archivo	LOC iniciales	LOC finales	Delta absoluto	% variacion
evaluador_alertas.js	473	502	+29	+6.1%
motor_reglas.js	55	74	+19	+34.5%
pac_gestor.js	1057	1068	+11	+1.0%

Cifras de `git diff --numstat` (398bf93 -> 24f29ed), archivo completo en los tres casos — atribuibles integramente a esta intervencion (ninguno de estos tres archivos fue tocado por otra intervencion de esta sesion).

Validacion y Pruebas Ejecutadas:

- ☒ Sintaxis validada con `node --check` (los 3 archivos)
- ☒ Auditoria con `/review` (verifico explicitamente: sin riesgo de deadlock, liberacion del lock garantizada en todos los `return` tempranos, sin doble adquisicion del mismo lock en la misma cadena de llamadas)
- ☐ Pruebas en navegador con `/qa` sobre URL de desarrollo (este modulo especifico — reglas y PAC — no se probó de forma aislada en el QA de 12.5; queda pendiente)

Impacto en Produccion: Elimina una condicion de carrera real sobre datos compartidos criticos (reglas de negocio, alertas activas, presupuesto PAC vigente). Hallazgo menor detectado en la auditoria (no bloqueante): un timeout de lock en `guardarReglasJSON` se reporta hoy con el mismo mensaje generico que un JSON invalido —

registrado como pendiente en `TODOS.md` (item 5).

12.4 [2026-08-05] [NS-01] RESOLUCION DE COLISION DE NAMESPACE `ONOPEN()`

Archivo(s) Intervenido(s):

- `MatrizSeguimiento_script/ImportarDato.js#L90`
- `normalizacion_script/MenuNormalizacion.js#L9`

Proposito del Archivo en el Sistema: Scripts auxiliares, historicamente desconectados del aplicativo principal (sin ninguna referencia cruzada desde `Codigo.js` / `pac_*.js` / `config.js` / `datos.js`), usados para importar y normalizar datos de origen antes de que lleguen a la matriz principal.

Motivo de la Intervencion: Ambos archivos declaraban su propia funcion `onOpen()`, que colisiona en el mismo namespace global de Apps Script con la `onOpen()` real de `Codigo.js` (linea 1843). Si estas carpetas llegaran a incluirse en un `clasp push` (el `.clasp.json` del proyecto no las excluye), la ultima declaracion cargada sobrescribiria silenciosamente a las otras, sin error visible.

Cambios Tecnicos Realizados:

- `onOpen()` -> `onOpenMatriz()` en `ImportarDato.js`.
- `onOpen()` -> `onOpenNormalizacion()` en `MenuNormalizacion.js`.

Fragmento de Codigo (Antes vs Despues):

`MatrizSeguimiento_script/ImportarDato.js#L90`:

```
// Antes
function onOpen() {
  LoggerSystema.info('Google Sheets abierto - Sistema iniciado v7.5 Compartido con IA');

// Despues
function onOpenMatriz() {
  LoggerSystema.info('Google Sheets abierto - Sistema iniciado v7.5 Compartido con IA');
```

`normalizacion_script/MenuNormalizacion.js#L9`:

```
// Antes
function onOpen() {
  SpreadsheetApp.getUi()
    .createMenu('Normalizacion IDU')

// Despues
function onOpenNormalizacion() {
  SpreadsheetApp.getUi()
    .createMenu('Normalizacion IDU')
```

Metricas del Cambio:

Metrica	Valor
LOC Anadidas	0
LOC Eliminadas	0
Delta absoluto	0
% variacion	0% (renombre de identificador, 1 linea por archivo, mismo numero de caracteres de linea)

Nota: ambos archivos aparecen como “creados” en el historial de git de este commit (nunca habian sido commiteados antes — `MatrizSeguimiento_script/ImportarDato.js` quedo en 1918 LOC totales y `normalizacion_script/MenuNormalizacion.js` en 158 LOC totales, ver Seccion 13), por lo que un `git diff --numstat` crudo mostraria el archivo completo como insertado. Esa cifra de archivo completo no representa el tamano real de esta intervencion, que fue puntual (1 palabra por archivo).

Validacion y Pruebas Ejecutadas:

- ☒ Búsqueda sobre todo el repositorio confirmando unica funcion `onOpen()` restante (la de `Codigo.js`) y cero referencias rotas a los nombres anteriores
- ☐ Auditoria con `/review` (cubierto de forma general en la revision de concurrencia de 12.3, no hubo pase dedicado a este cambio)
- ☐ Pruebas en navegador con `/qa` (estos archivos no forman parte del deployment probado en 12.5 — siguen sin integrarse al proyecto principal)

Impacto en Produccion: Elimina un riesgo de despliegue silencioso. Queda pendiente en `TODOS.md` la decision de fondo: donde deben vivir estas dos carpetas (proyecto Apps Script separado, exclusion via `.claspignore`, o integracion formal).

12.5 [2026-08-05] [FE-01] DESACOPLAMIENTO DE INDEX.HTML (CSS Y LOGICA JS PRINCIPAL)

Archivo(s) Intervenido(s):

- `Index.html#L24, L1595` (puntos de inclusion)
- `estilos.html` (archivo nuevo)
- `app_js.html` (archivo nuevo)

Proposito del Archivo en el Sistema: `Index.html` era un archivo monolitico de 7340 lineas que combinaba markup, un bloque `<style>` de mas de 1300 lineas y un bloque `<script>` de mas de 4200 lineas con las ~135 funciones de la interfaz.

Motivo de la Intervencion: Un archivo tan grande genera alto riesgo de conflictos de merge y dificulta ubicar codigo relacionado. Apps Script no tiene bundler nativo, pero `HtmlService`'s `include()` (ya usado para `pac_seccion.html`) permite dividir el archivo en partials sin cambiar la arquitectura de despliegue.

Cambios Tecnicos Realizados:

- Se extrajo el bloque `<style>` principal (1364 lineas) a `estilos.html`.
- Se extrajo el bloque `<script>` principal (4281 lineas, ~135 funciones) a `app_js.html`.

- `Index.html` quedo en 1697 lineas, enlazando ambos archivos via `<?!= include('estilos') ?>` y `<?!= include('app_js') ?>`.

Fragmento deCodigo (Antes vs Despues):

Antes (`Index.html` , estructura del `<head>`):

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/html2pdf.js/0.10.1/html2pdf.bundle.min.js">
</script>

<style>
  :root {
    --sidebar-bg: #2c3e50;
    ... (1362 lineas mas de CSS) ...
  }
</style>
```

Despues:

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/html2pdf.js/0.10.1/html2pdf.bundle.min.js">
</script>

<?!= include('estilos') ?>
```

Y de forma analoga para el bloque `<script>` principal (`Index.html` linea 1595 original): las ~4281 lineas de logica JS quedaron reemplazadas por `<?!= include('app_js') ?>`.

Metricas del Cambio:

Archivo	LOC iniciales	LOC finales	Delta absoluto	% variacion
<code>Index.html</code>	6898	1697	-5201	-75.4%
<code>estilos.html</code>	0 (no existia)	1364	+1364	N/A (archivo nuevo)
<code>app_js.html</code>	0 (no existia)	4281	+4281	N/A (archivo nuevo)

Cifras de `Index.html` verificadas por `git show <commit>:Index.html \ | wc -l` en los commits limite de la sesion (398bf93 -> 24f29ed) — incluyen tanto esta extraccion como el cambio de 12.2 (XSS) sobre el mismo archivo; el delta de -5201 esta dominado por la extraccion (~5643 de traslado neto) parcialmente compensado por las +11 lineas netas de 12.2 y otras diferencias menores de contexto entre ambos commits.

Validacion y Pruebas Ejecutadas:

- ☒ Sintaxis validada con `node --check` (JS extraido en `app_js.html` y bloque `<script>` remanente de `Index.html`)
- ☒ Auditoria con `/review` (confirmando que `include()` es concatenacion de texto en servidor — no ES modules, no iframes, no shadow DOM — por lo que las funciones globales de `app_js.html` siguen siendo accesibles desde atributos `onclick` y `google.script.run` exactamente igual que antes de la extraccion)

- ☒ Pruebas en navegador con `/qa` sobre URL de desarrollo: `clasp push` (37 archivos) a entorno de pruebas confirmado manualmente por el usuario como NO-produccion; QA manual en navegador real (Chrome, sesion autenticada del usuario) sobre la URL de deployment `@HEAD` — sin div de error de `include()` faltante, CSS cargado correctamente, sin `ReferenceError` en consola, botones `onclick` de la matriz responden, guardado de seguimiento funciona, modulo de alertas carga, modulo PAC funciona. **QA aprobado.**

Impacto en Produccion: Reduce el archivo principal de interfaz de 7340 a 1697 lineas. Pendiente en `TODOS.md`: sub-dividir `app_js.html` (hoy un solo archivo de 4281 lineas) por seccion funcional (matriz, PAC, alertas, auditoria, permisos), que era el alcance completo originalmente planteado.

12.6 CORRECCION SOBRE DISPONIBILIDAD DE `MAKE-PDF` (VER SECCIONES 3.3 Y 10)

Las Secciones 3.3 y 10 de este documento (escritas por la sesion anterior) indicaban que el binario local de `/make-pdf` no estaba disponible. Verificacion independiente del 2026-08-05: **el binario si existe** (`~/claude/skills/gstack/make-pdf/dist/pdf.exe`) y soporta salida directa a `.docx` (`--to docx`), sin necesitar Pandoc instalado por separado. Sin embargo, `pdf.exe setup` falla hoy en este equipo por un problema del componente de navegador que usa internamente (`browse newtab` rechaza la URL `about:blank` con “scheme not allowed”), y no se resuelve con un `./setup` completo de gstack. Es decir: la herramienta esta instalada y su CLI responde, pero la renderizacion real a PDF/DOCX esta bloqueada hoy por un problema de entorno — distinto del diagnostico original (“no esta disponible”).

Actualizacion 2026-08-05 (segunda verificacion, mismo dia): el diagnostico de “renderizacion bloqueada” tambien quedo parcialmente corregido. `pdf.exe setup` (el smoke test interno de la herramienta) sigue fallando con el mismo error de Chromium, pero el comando real de uso, `pdf.exe generate <archivo.md> <salida> --to docx|pdf`, **funciona correctamente para ambos formatos** cuando se ejecuta directamente sobre este mismo documento: `Documento_Tecnico_Aplicacion_Predios.docx` (6810 palabras, 529KB, generado en 0.6s) y `Documento_Tecnico_Aplicacion_Predios.pdf` (con `--cover --toc`, 6810 palabras, 1023KB, generado en 9.3s, incluyendo el paso de Chromium que fallaba en `setup`). Conclusion revisada: la exportacion formal a Word y PDF **si esta operativa** en este equipo; el fallo de `setup` es un problema aislado del propio smoke test, no de la ruta de generacion real. Se retira el item 6 de `TODOS.md` como resuelto.

12.7 [2026-08-05] [CONC-P2.1] DIFERENCIAR TIMEOUT DE LOCK VS JSON INVALIDO EN MOTORES DE REGLAS

Archivo(s) Intervenido(s):

- `evaluador_alertas.js#L8-L86`
- `motor_reglas.js#L8-L86`

Proposito del Archivo en el Sistema: `evaluador_alertas.js` y su copia duplicada `motor_reglas.js` exponen `obtenerReglasJSON()` / `guardarReglasJSON()`, usadas por la UI de administracion para leer y escribir el JSON maestro de reglas de negocio (hoja oculta `CONFIG_REGLAS`).

Motivo de la Intervencion: Hallazgo [P2] de la auditoria de Staff Engineer registrado en `TODOS.md` (item 5) tras la intervencion [CONC-P2] (12.3): en `guardarReglasJSON`, `lock.waitLock(20000)` vivia dentro del mismo `try` que `JSON.parse(jsonString)`. Si el lock expiraba por contencion (dos administradores guardando reglas casi al mismo tiempo), el unico `catch` devolvía el mensaje generico “El formato JSON es invalido” — un diagnostico falso para

un admin cuyo JSON si era correcto. En `obtenerReglasJSON`, un timeout de lock durante la auto-creacion de `CONFIG_REGLAS` degradaba a un `TypeError` capturado por el catch externo, con mensaje tecnico en vez de explicar la causa real.

Cambios Tecnicos Realizados:

- `guardarReglasJSON`: `JSON.parse(jsonString)` se valida en un `try/catch` propio **antes** de tocar el lock — un JSON invalido nunca llega a `lock.waitLock()`.
- `guardarReglasJSON`: la adquisicion del lock pasa a su propio `try/catch` independiente; si `waitLock(20000)` lanza excepcion, retorna `{ success: false, message: 'El sistema se encuentra ocupado por otro administrador. Por favor intente de nuevo en unos segundos.' }` sin ejecutar la logica de guardado.
- `obtenerReglasJSON`: el `catch` del bloque de auto-creacion de `CONFIG_REGLAS` retorna el mismo mensaje de “sistema ocupado” en vez de dejar que degrade a un `TypeError` sin contexto.
- `lock.releaseLock()` se mantiene garantizado (via `finally`) en toda ruta donde el lock fue realmente adquirido; las rutas donde `waitLock()` lanza excepcion no liberan porque el lock nunca llego a adquirirse (no hay fuga).
- Cambio aplicado de forma identica en ambos archivos (duplicacion intencional preexistente, no se corrige aqui).

Fragmento deCodigo (Antes vs Despues):

Antes (`evaluador_alertas.js` / `motor_reglas.js`, `guardarReglasJSON`):

```
function guardarReglasJSON(jsonString, usuario) {
  try {
    JSON.parse(jsonString); // Si esto falla, tira error abajo
  } catch (e) {
    return { success: false, error: "El formato JSON es inválido. Revisa la sintaxis." };
  }

  const lock = LockService.getScriptLock();
  try {
    lock.waitLock(20000);
    const fileId = getConfig('DATA_FILES.PRINCIPAL');
    ...
  }
```

Despues:

```
function guardarReglasJSON(jsonString, usuario) {
  // ✅ CONC-P2.1: validar el JSON ANTES de tocar el lock, para que un JSON invalido
  // nunca se reporte como timeout ni un timeout se reporte como JSON invalido
  try {
    JSON.parse(jsonString);
  } catch (parseError) {
    return { success: false, error: "El formato JSON es inválido. Revisa la sintaxis." };
  }

  const lock = LockService.getScriptLock();
  try {
    lock.waitLock(20000);
  } catch (lockError) {
    return { success: false, message: 'El sistema se encuentra ocupado por otro administrador. Por favor
    intente de nuevo en unos segundos.' };
  }

  try {
    const fileId = getConfig('DATA_FILES.PRINCIPAL');
    ...
  } finally {
    try { lock.releaseLock(); } catch (er) {}
  }
}
```

Metricas del Cambio:

Archivo	LOC iniciales	LOC finales	Delta absoluto	% variacion
evaluador_alertas.js	502	513	+11	+2.2%
motor_reglas.js	74	85	+11	+14.9%

Cifras verificadas con `git diff --numstat` contra HEAD (+15/-4 lineas en ambos archivos) — atribuibles integramente a esta intervencion puntual (ningun otro cambio toco estos dos archivos en esta sesion despues de 12.3).

Validacion y Pruebas Ejecutadas:

- ✅ Sintaxis validada con `node --check` (ambos archivos, OK)
- ✅ Auditoria como Staff Engineer (equivalente a `/review`, repo sin rama base): confirmado que `lock.releaseLock()` se sigue ejecutando en toda ruta donde el lock fue realmente adquirido, que no existe ninguna ruta de adquisicion-sin-liberacion, que el comportamiento del camino exitoso (JSON valido + lock disponible + escritura correcta) no cambio, y que ambos archivos quedan comportamentalmente identicos. Hallazgo menor no bloqueante: en `obtenerReglasJSON`, el `catch` del bloque de auto-creacion tambien captura errores no relacionados con el lock (p. ej. un fallo real de `insertSheet`), heredado de la intervencion [SEC-P1.5] previa a esta sesion — no es una regresion de este cambio y queda fuera de alcance (afecta solo el escenario raro de primera ejecucion sin hoja `CONFIG REGLAS`).
- ❑ Pruebas en navegador con `/qa` sobre URL de desarrollo (cambio en manejo de errores de backend; no se ejecuto una prueba de UI dedicada para forzar contencion de lock real en esta intervencion)

Impacto en Produccion: Un administrador que intenta guardar reglas mientras otro tiene el lock ahora ve un mensaje que describe correctamente la causa (“sistema ocupado, intente de nuevo”) en vez de un mensaje de JSON invalido que lo llevaria a revisar sin sentido un JSON que ya era correcto. Cierra el ítem 5 de `TODOS.md` como `[COMPLETADO]`.

12.8 [2026-08-05] [PLAN-FE-02] DICTAMEN DE ARQUITECTURA — SUBDIVISIÓN MODULAR DE APP_JS.HTML

Archivo(s) Intervenido(s):

- Ninguno modificado en código. Análisis de lectura sobre `app_js.html` (4281 líneas completas) e `Index.html#L1595` (punto de `include()` actual).

Propósito del Archivo en el Sistema: `app_js.html` es el partial extraído en FE-01 (12.5) que concentra toda la lógica JS de la interfaz (matriz, reglas/alertas, permisos/auditoría) en un único `<script>` de 4281 líneas.

Motivo de la Intervención: Seguimiento planificado del ítem 3 de `TODOS.md` (FE-01): la extracción de `app_js.html` resolvió el riesgo de merge del `Index.html` original de 7340 líneas, pero dejó 116 funciones de 4 dominios de negocio distintas mezcladas en un solo archivo. El usuario solicitó, vía `/plan-eng-review`, un dictamen de arquitectura para subdividirlo en `app_core_js.html` / `app_matriz_js.html` / `app_alertas_js.html` / `app_permisos_js.html` antes de ejecutar el corte.

Metodología (por qué no se reutilizaron los números de la Sección 12.5): Un `grep '^function'` ingenuo subcuenta las funciones top-level porque ~15 están indentadas por formato inconsistente aunque su profundidad de llaves real es 0. Se recorrió el archivo completo rastreando profundidad de `{ / }` para ubicar con precisión cada declaración top-level, luego se midió el uso real de `rawData` / `currentData` / `currentUser` / `state` y `google.script.run` por función (no estimado), y se verificó por `grep` dirigido que no existan llamadas cruzadas entre los 3 módulos hoja propuestos.

Cambios Técnicos Realizados:

- Ninguno en código — este es un entregable de planificación. Ver `TODOS.md` ítem 7 para el detalle completo de la clasificación función-por-función y la secuencia de extracción recomendada.
- Se corrigieron los números citados en la Sección 12.5 (ver tabla abajo).
- Se identificó y documentó un bug preexistente no relacionado (`openModal()` invocado en `app_js.html:1936` y `:2585` pero nunca definido en el repo) como hallazgo colateral, sin corregirlo.

Números corregidos (12.5 → verificación real 2026-08-05):

Métrica	Sección 12.5 (estimado)	Verificado hoy (por profundidad de llaves)
Funciones top-level	~135	127 declaraciones, 116 nombres únicos
Funciones duplicadas	4 (<code>saveNavigationState</code> , <code>restoreNavigationState</code> , <code>setupModalCleanup</code> , <code>onTramoChange</code>)	11 (+ <code>showToast</code> , <code>showNotification</code> , <code>showConfirmation</code> , <code>confirmAction</code> , <code>onError</code> , <code>populateDropdowns</code> , <code>onProyectoChange</code>)
<code>google.script.run</code>	~34	33

Clasificación por módulo (evidencia, no estimación):

Destino	Funciones	LOC reales	state	currentUser	rawData / currentData
app_core_js.html	31	~861-999*	0	uso transversal (declara + usa)	declara + onDataLoaded / abrirNormalizacion lo leen
app_matriz_js.html	56	2390	101 (~100% del total)	uso puntual	uso casi exclusivo
app_alertas_js.html	16	508	0	uso puntual	0
app_permisos_js.html	24	364	0	uso puntual	0

* Rango según si las 11 duplicadas se reconcilian a una sola copia antes de mover el código (~861) o se copian ambas temporalmente (~999).

Grafo de dependencias (resumen; ver respuesta completa en la conversación para el ASCII detallado):

app_core_js.html es la única raíz — declara todas las variables `let/const` de nivel superior una sola vez (obligatorio: GAS no tiene scope por partial, todo `include()` cae en el mismo `<script>` global, y una redeclaración `let` duplicada entre archivos es un `SyntaxError` que rompe la carga completa de la página). Los 3 módulos hoja (`matriz`, `alertas`, `permisos`) solo leen/escriben esas variables y utilidades de core; **cero llamadas cruzadas entre ellos**, confirmado por `grep` dirigido en ambos sentidos.

Orden de `include()` recomendado (`Index.html#L1595`, reemplazando `<?!= include('app_js') ?>`):

```
<?!= include('app_core_js') ?>
<?!= include('app_matriz_js') ?>
<?!= include('app_alertas_js') ?>
<?!= include('app_permisos_js') ?>
```

app_core_js debe ir primero siempre. El orden entre los otros 3 no importa (sin dependencias entre ellos). El riesgo real de `ReferenceError` por orden incorrecto es bajo hoy porque todo el consumo de globals ocurre dentro de cuerpos de función o del único `$(document).ready` (línea 320, va en core, se dispara después de que los 4 `<script>` ya cargaron) — pero el orden sigue siendo la práctica correcta y defensiva ante código futuro que ejecute algo a nivel superior.

Secuencia de extracción recomendada: `app_core_js.html` → `app_alertas_js.html` (módulo más chico, canario de bajo riesgo) → `app_permisos_js.html` → `app_matriz_js.html` al final (el más grande, 56% del archivo, y el único con lógica transaccional de guardado — `submitTracking` → `saveFollowupData`).

Validación y Pruebas Ejecutadas:

- ☒ Análisis de código real (no memoria de sesión): profundidad de llaves, conteo de globals por función, `grep` dirigido de llamadas cruzadas entre módulos propuestos
- ☐ Sintaxis validada con `node --check` (no aplica — no se generó código en esta intervención)
- ☐ Pruebas en navegador con `/qa` (no aplica — planificación, no ejecución)

Impacto en Producción: Ninguno todavía — es un plan, no una ejecución. Reduce el riesgo de la futura ejecución al reemplazar suposiciones (“~135 funciones”, “4 duplicadas”) por conteos verificados y un grafo de dependencias con evidencia de que los 3 módulos hoja son realmente independientes entre sí. Descubrió como efecto colateral un bug de referencia rota preexistente (`openModal` no definida) que queda registrado pero no corregido.

12.9 [2026-08-05] [CONC-FE-02] SUBDIVISIÓN MODULAR DE APP_JS.HTML — FASES 1 Y 2 (BUILD)

Archivo(s) Intervenido(s):

- `app_core_js.html` (archivo nuevo)
- `app_alertas_js.html` (archivo nuevo)
- `app_js.html#L1-L4281` (reducido progresivamente)
- `Index.html#L1595-L1597` (directivas `include()`)

Propósito del Archivo en el Sistema: Ejecución del plan documentado en 12.8: dividir `app_js.html` (el parcial monolítico de 4281 líneas surgido de FE-01) en módulos por dominio funcional, manteniendo el mismo modelo de despliegue basado en `include()`.

Motivo de la Intervención: Seguimiento directo de la planificación 12.8 / ítem 7 de `TODOs.md`. El usuario autorizó la ejecución en dos fases secuenciales de bajo riesgo, empezando por la base (`core`) y el módulo más pequeño y aislado (`alertas`), antes de tocar los módulos más grandes (`permisos` , `matriz`).

Cambios Técnicos Realizados — Fase 1 (`app_core_js.html`):

- Extraídas las 16 declaraciones `let/const` globales listadas en el plan (`rawData` , `currentData` , `currentUser` , `currentRole` , `state` , `C` , `navigationState` , `motorReglasData` , `alertasGlobales` , `filtroNivelActivo` , `seguimientoData` , `originalData` , `currentDropdowns` , `pdfDataStructure` , `searchableSelectState` , `searchableSelectConfig`) — cada una declarada una única vez, ahora solo en este archivo.
- Extraídas 23 funciones (utilidades UI, parseo, bootstrap de datos, navegación).
- Reconciliadas las 8 parejas de funciones duplicadas que le correspondían a este dominio (`setupModalCleanup` , `saveNavigationState` , `restoreNavigationState` , `showToast` , `showNotification` , `showConfirmation` , `confirmAction` , `onError`).
- `searchableSelectConfig.onCommit` reescrito con wrappers de función flecha (`(...args) => onProyectoChange(...args)`) en vez de referencias directas, para evitar un `ReferenceError` por referencia adelantada (`onProyectoChange` / `onTramoChange` / `applyFilters` viven en `app_js.html` , que carga *después* de `app_core_js.html`).

Decisión de reconciliación no trivial — `setupModalCleanup`: la copia que estaba activa en producción (la última declarada, por las reglas de shadowing de JS) era una versión *menos completa* que la copia descartada: omitía la limpieza de `#rtDetailModal` y el reset de `originalData` al cerrar el modal de tracking. Se confirmó explícitamente con el usuario (AskUserQuestion) antes de conservar la versión más completa en vez de la “última declarada” — es un cambio de comportamiento real (restaura limpieza que hoy no ocurre), no solo un refactor de ubicación.

Cambios Técnicos Realizados — Fase 2 (`app_alertas_js.html`):

- Extraídas las 16 funciones del dominio de reglas de negocio y alertas tempranas: `cargarJSONenPantalla`, `renderizarTarjetasReglas`, `renderizarFuentesDeDatos`, `abrirEditorVisual`, `guardarEdicionReglaVisual`, `guardarJSONdesdePantalla`, `formatearJSON`, `ejecutarMotorManual`, `cargarAlertasWeb`, `renderizarAlertas`, `popularDropdownsAlertas`, `filtrarAlertasPorNivel`, `limpiarFiltrosAlertas`, `aplicarFiltrosAlertas`, `exportarAlertasExcel`, `programarReporte`.
- El handler de evento `$(document).on('click', '.menu-item', ...)` y el bloque `$(document).ready(...)` de inicialización, aunque están físicamente entre las funciones movidas, se dejaron intencionalmente en `app_js.html` — no forman parte de la lista explícita de 16 funciones y no requieren moverse (el nuevo orden de `include()` los deja seguros de todas formas).

Orden de `include()` en `Index.html` (líneas 1595-1597):

```
<?!= include('app_core_js') ?>
<?!= include('app_alertas_js') ?>
<?!= include('app_js') ?>
```

Métricas del Cambio:

Archivo	LOC iniciales	LOC finales	Delta absoluto
<code>app_js.html</code>	4281 (antes de Fase 1)	2952 (después de Fase 2)	-1329 (-31.0%)
<code>app_core_js.html</code>	0 (no existía)	698	+698 (archivo nuevo)
<code>app_alertas_js.html</code>	0 (no existía)	502	+502 (archivo nuevo)

Desglose por fase: Fase 1 redujo `app_js.html` de 4281 a 3450 (-831 líneas, incluye la eliminación neta de las 8 duplicadas reconciliadas). Fase 2 redujo de 3450 a 2952 (-498 líneas).

Validación y Pruebas Ejecutadas:

- ☒ Sintaxis validada con `node --check` sobre el JS interno de cada parcial (`app_core_js.html`, `app_alertas_js.html`, `app_js.html`), en cada fase
- ☒ Auditoría de integridad: cero funciones duplicadas nuevas entre los 3 archivos (la única colisión detectada, `onTramoChange`, es preexistente y está contenida enteramente dentro de `app_js.html`, pendiente para la Fase 4); las 16+23 funciones movidas verificadas como presentes exactamente una vez, solo en su archivo de destino
- ☒ `clasp push` (39 archivos) al entorno de pruebas ya confirmado como NO-producción, ejecutado para que la URL `@HEAD` sirva el código de las Fases 1 y 2
- ☐ **QA manual en navegador real: solicitado al usuario (checklist de consola/ReferenceError + módulo de reglas + tarjetas de alertas), sin confirmación de resultado recibida al momento de este cierre de sesión.** No se marca como verificado hasta recibir esa confirmación explícita.

Impacto en Producción: `app_js.html` pasó de 4281 a 2952 líneas (-31%) en dos pasos de bajo riesgo.

`app_core_js.html` y `app_alertas_js.html` están desplegados en el entorno de pruebas pero **su verificación funcional en navegador real está pendiente** — no debe asumirse que Fase 1/2 están completamente cerradas hasta esa confirmación.

12.10 [2026-08-06] [CONC-FE-02] SUBDIVISIÓN MODULAR DE APP_JS.HTML — FASE 3 (BUILD) — CIERRE ACUMULADO FASES 1-3

Archivo(s) Intervenido(s):

- `app_permisos_js.html` (archivo nuevo)
- `app_js.html#L2140-L2502` (bloque removido)
- `Index.html#L1595-L1598` (directiva `include()` añadida)

Propósito del Archivo en el Sistema: Continuación directa de 12.9 (Fases 1-2). `app_permisos_js.html` concentra el dominio de gestión de usuarios, roles, reportes guardados, historial y auditoría — antes mezclado en `app_js.html` junto con matriz.

Cambios Técnicos Realizados:

- Extraídas las 24 funciones del dominio de permisos/reportes/auditoría: `applyHistorialFilters`, `onHistorialLoaded`, `applyAuditoriaFilters`, `onAuditoriaLoaded`, `openPermissionModal`, `savePermission`, `onPermissionSaved`, `loadPermissions`, `onPermissionsLoaded`, `deletePermission`, `onPermissionDeleted`, `createNewReport`, `generateReport`, `onReportGenerated`, `loadReports`, `onReportsLoaded`, `downloadReport`, `deleteReport`, `onReportDeleted`, `validateIntegrity`, `onIntegrityValidated`, `clearCache`, `exportSystemInfo`, `onSystemInfoExported`.
- Bloque contiguo sin código intercalado de otros dominios (a diferencia de Fase 2, aquí no hubo que preservar ningún handler de evento en medio).
- Orden de `include()` en `Index.html` (líneas 1595-1598): `app_core_js` → `app_alertas_js` → `app_permisos_js` → `app_js`.

Métricas del Cambio (acumulado Fases 1-3):

Archivo	LOC	Nota
<code>app_js.html</code>	4281 → 3450 → 2952 → 2591	-1690 LOC (-39.5%) acumulado sobre el original
<code>app_core_js.html</code>	698	Fase 1
<code>app_alertas_js.html</code>	502	Fase 2
<code>app_permisos_js.html</code>	371	Fase 3 (nuevo)

Desglose Fase 3: `app_js.html` de 2952 a 2591 (-361 líneas).

Validación y Pruebas Ejecutadas:

- ☒ Sintaxis validada con `node --check` sobre los 4 parciales activos (`app_core_js.html`, `app_alertas_js.html`, `app_permisos_js.html`, `app_js.html`)
- ☒ Auditoría de integridad: las 24 funciones de permisos verificadas presentes exactamente una vez, solo en `app_permisos_js.html`; cero referencias a globals de `alertas` (`motorReglasData` / `alertasGlobales` / `filtroNivelActivo`) ni llamadas a funciones de `matriz`; los 7 IDs del DOM que usa (`permissionModal`, `reportModal`, `historialBody`, `auditoriaBody`, `permisosBody`, `reportesContainer`, `detailModal`) confirmados presentes en `Index.html`. Única colisión de nombre en el repo: `onTramoChange`, preexistente, contenida enteramente en `app_js.html`, pendiente de la Fase 4

- ☒ `clasp push` (40 archivos) al entorno de pruebas confirmado NO-producción, para que `@HEAD` sirva el código de las Fases 1-3
- ☐ **QA manual en navegador real: sigue pendiente.** Se pidió dos veces (cierre de Fase 2 y cierre de Fase 3) y aún no se recibió confirmación de resultado del usuario. No se marca como verificado — las Fases 1-3 están desplegadas en `@HEAD` pero su comportamiento en navegador real no ha sido confirmado en esta sesión.

Impacto en Producción: `app_js.html` acumula una reducción de 1690 líneas (-39.5%) sobre el original de 4281, repartidas en 3 extracciones de bajo riesgo (core → alertas → permisos), todas con cero llamadas cruzadas entre los módulos hoja creados hasta ahora. Queda pendiente la Fase 4 (`app_matriz_js.html`, el módulo más grande, ~2390 LOC, y el único que exige reconciliar duplicados de matriz) y, transversalmente, la verificación en navegador real de las 3 fases ya desplegadas.

13. Inventario Exhaustivo del Repositorio

Inventario factual archivo por archivo de los 23 archivos que componen el núcleo del backend, el frontend y los subproyectos auxiliares. Elaborado el 2026-08-05 leyendo directamente el código fuente (no inferido de nombres de archivo). Los archivos de los subproyectos `MatrizSeguimiento_script/` y `normalizacion_script/` se confirmaron 100% independientes del backend principal: cero referencias a `getConfig()`, `GestorDatos`, `GestorPermisos` o `GestorAuditoria` en ninguno de sus 11 archivos.

13.1 NUCLEO DEL BACKEND

Codigo.js (1985 LOC) — Orquestador principal: `doGet` (punto de entrada web), `saveFollowupData` / `saveTrackingData` (registro de seguimiento), generación de reportes PDF, gestión de filtros de matriz y utilidades de staging (promoción Dato2->Dato1). 48 funciones de nivel superior. Sin variables globales de nivel superior. Dependencias: `getConfig()` x44, `new GestorPermisos()` x10, `new GestorDatos()` x5, `new GestorAuditoria()` x5, `new GestorReportes()` x4. **Hallazgo de arquitectura:** `getPermissionsData`, `savePermission`, `deletePermission`, `getUserRole`, `getAllowedProjects` (L864-919) y `getSavedReports`, `saveReport`, `executeReport`, `deleteReport` (L927-976) son wrappers globales con el mismo nombre e idéntica firma que funciones ya definidas en `permisos.js` y `reportes.js` — 9 funciones duplicadas en 3 archivos distintos del mismo namespace global de Apps Script.

config.js (872 LOC) — Configuración central: objeto `const CONFIG` (L7, único global del archivo) y accesor `getConfig(path, default)` (L190) por ruta de puntos, más `validateConfig`, `diagnosticarSistema` y 20 helpers derivados. No depende de ningún otro archivo del proyecto (solo servicios nativos de Apps Script) — es la base de la que dependen casi todos los demás.

permisos.js (302 LOC) — Clase `GestorPermisos`: `obtenerRol` (L21), `obtenerProyectos` (L48), `obtenerTodos` (L75), `guardarPermiso` (L108), `eliminarPermiso` (L169), `obtenerHistorialPermisos` (L234). Más 5 funciones globales wrapper (L255-299) que duplican a las de `Codigo.js` señaladas arriba. Dependencias: `getConfig()` x15, `new GestorDatos()` x1, `new GestorAuditoria()` x1, `LockService` x1.

reportes.js (310 LOC) — Clase `GestorReportes`: CRUD de reportes guardados por usuario (`obtenerGuardados`, `guardarReporte`, `ejecutarReporte`, `eliminarReporte`). 4 funciones globales wrapper (L264-300) que duplican a las de `Codigo.js`. Dependencias: `getConfig()` x12, `new GestorDatos()` x1, `new GestorAuditoria()` x1, `LockService` x2.

cache_backend.gs (327 LOC) — Backend RBAC y de cache asincrono, destino de las llamadas

`google.script.run` de `Index.html`: `getUserRoles` (L23), `forceCacheInvalidation` (L90), `processCacheQueue` (L138, worker de una cola en la hoja `CacheQueue`), `getSearchHints` (L251). Dependencias: `getConfig` (x17, `new GestorDatos`(x2, `new GestorPermisos`(x2, `LockService` x3).

evaluador_alertas.js (502 LOC) — Motor de reglas de negocio: `obtenerReglasJSON` / `guardarReglasJSON` (L8-75)

y clase `MotorEvaluadorReglas` (`ejecutarMotor`, `_cumpleCondiciones`, `_evaluarReglaTiempo`, `_guardarAlertasEnHoja`, calculo de dias habiles con festivos de Colombia via `CalendarApp`). Dependencias: `getConfig`(x5, `new GestorAuditoria`(x1, `LockService` x6. **Hallazgo de arquitectura (ya conocido, confirmado de nuevo):** L8-75 son codigo identico, caracter por caracter, al contenido integro de `motor_reglas.js`.

motor_reglas.js (74 LOC) — Duplicado exacto de `obtenerReglasJSON` / `guardarReglasJSON` de

`evaluador_alertas.js` L8-75. Dependencias: `getConfig`(x2, `new GestorAuditoria`(x1, `LockService` x4.

pac_gestor.js (1068 LOC) — Motor del subsistema PAC: lectura de hojas externas, calculo de semaforo presupuestal (`calcularSemaforoPAC`), recomendaciones de reemplazo de RT, y flujo vigente/borrador

(`sincronizarPAC`, `aprobarBorradorPAC`, `rechazarBorradorPAC`, `_pac_compararYGenerarBorrador`). No usa `getConfig`(ni clases `Gestor*` — depende de simbolos propios del modulo PAC (`PAC_CONFIG` x26, `pac_log`(x21, `pac_getSpreadsheet`(x8) definidos en otros archivos `pac_*` fuera de este inventario. `LockService` x4.

Hallazgo de arquitectura: `pac_leerHojaExterna` (L9) crea implicitamente una variable global no declarada `_PAC_RUNTIME_CACHE` (`if (typeof _PAC_RUNTIME_CACHE === 'undefined') _PAC_RUNTIME_CACHE = {};`) usada como cache de ejecucion en varias funciones del archivo.

busqueda.js (15 LOC) — Pese al nombre, no es un modulo de busqueda: es una funcion de diagnostico manual (`testFinal`) que ejecuta pruebas de humo llamando a `getUserAndRole` y `getDashboardData` (ambas definidas en `Codigo.js`). No forma parte del flujo de produccion.

13.2 FRONTEND (INDEX.HTML Y LOS DOS ARCHIVOS EXTRAIDOS EN FE-01)

Index.html (1697 LOC) — Plantilla principal servida por `doGet`: estructura Bootstrap de la SPA, tres directivas

`include()` (`estilos`, `pac_seccion`, `app_js`) y un bloque `<script>` inline (L875-1072) con utilidades de cache/actualizacion forzada: `LocalCache` (clase, L901), `gsForceCacheInvalidation`, `gsGetSearchHints`, `initializeApp`, `handleForceUpdate`. Variables globales del script inline: `const COST_THRESHOLD = 5.00` (L917), `let currentUserRoles = []` (L918). `google.script.run` x4, apuntando a `cache_backend.gs`.

estilos.html (1364 LOC) — Hoja de estilos CSS pura (un unico bloque `<style>`). Sin JS, sin variables globales, sin `google.script.run`. Incluido por `Index.html` via `include('estilos')`.

app_js.html (4281 LOC) — El modulo mas grande del proyecto: toda la logica de interaccion de la SPA

(dashboard, filtros, matriz, edicion del motor de reglas, gestion de permisos/reportes/auditoria desde la UI, exportacion Excel/PDF, paginacion, notificaciones, "Filtro Matriz"). 60 funciones detectadas en columna 0. 21 variables/objetos de estado global de nivel superior (`rawData`, `allColsList`, `currentData`, `tableInstance`, `currentUser`, `currentRole`, `state`, `motorReglasData`, `alertasGlobales`, etc.). `google.script.run` x34, apuntando a funciones de `Codigo.js`, `permisos.js`, `reportes.js`, `motor_reglas.js` / `evaluador_alertas.js`.

Hallazgo de arquitectura: existen 4 funciones redeclaradas dentro del propio archivo con el mismo nombre (`saveNavigationState`, `restoreNavigationState`, `setupModalCleanup`, `onTramoChange`) — la segunda declaracion sobrescribe silenciosamente a la primera en el mismo scope global del navegador, sin error de JavaScript.

13.3 SUBPROYECTO MATRIZSEGUIMIENTO_SCRIPT/ (INDEPENDIENTE DEL BACKEND PRINCIPAL)

CopiarDatosConsolidado.js (339 LOC) — Importa datos desde `Consolidado_Script` hacia `Datos2`, respetando un rango de columnas intocables (E:R) via mapeo dinamico de encabezados. `var CFG` (L4) es su unico global. Autocontenido, sin dependencias externas al archivo.

Festivos.js (55 LOC) — Importa el calendario oficial de festivos de Colombia desde `CalendarApp` hacia una hoja `FESTIVOS`. Usa `LoggerSistema` (definido en `ImportarDato.js`, mismo directorio).

ImportarDato.js (1918 LOC) — El archivo mas grande del subproyecto: sistema de “cruces” configurables (importacion/mapeo desde URLs externas), triggers programados, clase `LoggerSistema` (logger propio con hoja de logs), y clase `MotorAprendizajeRT` (aprendizaje de coincidencias aproximadas de claves RT, persistido en `PropertiesService`). Contiene `onOpenMatriz` (L90, renombrada en NS-01). 7 constantes globales (`CONFIG_PROP`, `PROGRAMACIONES_PROP`, `AUDIT_SHEET`, `LOG_SHEET`, `HISTORY_SHEET`, `TEMP_SHEET_PREFIX`, `BATCH_SIZE`). Cero referencias a `getConfig()` o clases `Gestor*` del proyecto principal.

obtenerEncabezados.js (6 LOC) — Utilidad minima de depuracion manual (`verEncabezados`) que registra en log los encabezados de la hoja `Datos` activa.

13.4 SUBPROYECTO NORMALIZACION_SCRIPT/ (INDEPENDIENTE DEL BACKEND PRINCIPAL)

ConfigNormalizacion.js (440 LOC) — Configuracion central del pipeline de normalizacion v6.0: `var CONFIG_NORMALIZACION` (L11) con hojas origen/destino y la “estructura objetivo” (~100 columnas destino).

CoreNormalizacion.js (832 LOC) — Motor ETL: normalizacion de encabezados, tipado estricto (fecha/numero/moneda/texto), completado de TRAMO desde PROYECTO, y unificacion de columnas duplicadas con logicas especificas por regla (`aplicarLogicaAceptaron`, `aplicarLogicaEstadoDelAvaluo`, etc.). 26 funciones. Depende de `CONFIG_NORMALIZACION` (x10).

ImportarConsolidado.js (704 LOC) — Importador de CSV via conversion temporal a Sheets, con selector interactivo de archivos en Drive. Usa su propio `var CONFIG_IMPORTADOR` (L26), no depende de `CONFIG_NORMALIZACION`.

MenuNormalizacion.js (158 LOC) — Orquestador del pipeline de 8 fases y menu “Normalizacion IDU”. Contiene `onOpenNormalizacion` (L9, renombrada en NS-01). Llama a funciones de `CoreNormalizacion.js`, `ReportesNormalizacion.js` y `UtilidadesNormalizacion.js`.

prueba.js (170 LOC) — Scripts de verificacion manual, no de produccion (`verificarUnificacionSolicitudAvaluo`, `diagnosticarREGLA_018`).

ReportesNormalizacion.js (179 LOC) — Genera la hoja `DATOS_NORMALIZADOS` con formato/resaltado de conflictos, y reportes de unificacion/convenciones/RTs problematicos/columnas similares.

UtilidadesNormalizacion.js (107 LOC) — Biblioteca auxiliar transversal: similitud de texto (Levenshtein) para detectar columnas duplicadas, utilidades de hoja, formateo de timestamp/duracion.

14. Matriz Consolidada de Metricas deCodigo

Tabla comparativa de LOC pre/post para los archivos tocados directamente por las intervenciones de la Seccion 12, verificada con `git show <commit>:<archivo> | wc -l` en los commits limite de la sesion (`398bf93` antes, `24f29ed` despues).

Archivo	LOC inicial	LOC final	Delta absoluto	% variacion	Intervencion(es)
<code>Codigo.js</code>	1863	1985	+122	+6.5%	SEC-P0 (parcial — ver nota)
<code>Index.html</code>	6898	1697	-5201	-75.4%	SEC-P1 + FE-01
<code>evaluador_alertas.js</code>	473	502	+29	+6.1%	CONC-P2
<code>motor_reglas.js</code>	55	74	+19	+34.5%	CONC-P2
<code>pac_gestor.js</code>	1057	1068	+11	+1.0%	CONC-P2
<code>estilos.html</code>	0 (no existia)	1364	+1364	N/A	FE-01
<code>app_js.html</code>	0 (no existia)	4281	+4281	N/A	FE-01
<code>MatrizSeguimiento_script/ImportarDato.js</code>	0 (no trackeado en git)	1918	+1918	N/A	NS-01 (solo 1 linea de contenido real)
<code>normalizacion_script/MenuNormalizacion.js</code>	0 (no trackeado en git)	158	+158	N/A	NS-01 (solo 1 linea de contenido real)

Nota de honestidad metodologica: las filas de `Codigo.js` e `Index.html` combinan mas de una intervencion sobre el mismo archivo en el mismo commit — el delta de archivo completo NO debe leerse como el tamano de una sola intervencion. Ver el desglose por intervencion en la Seccion 12 (12.1, 12.2 y 12.5) para las cifras acotadas a cada cambio especifico. Las filas de `MatrizSeguimiento_script/ImportarDato.js` y `normalizacion_script/MenuNormalizacion.js` muestran el archivo completo como “insertado” porque nunca habian sido commiteados antes de esta sesion — la intervencion real (NS-01) fue de una sola palabra por archivo, no de miles de lineas.

Totales agregados de la sesion (Seccion 12, commit `24f29ed`):

- Archivos modificados directamente por las 5 intervenciones documentadas: 7 (`Codigo.js`, `Index.html`, `evaluador_alertas.js`, `motor_reglas.js`, `pac_gestor.js`, `MatrizSeguimiento_script/ImportarDato.js`, `normalizacion_script/MenuNormalizacion.js`).
- Archivos nuevos creados: 2 (`estilos.html`, `app_js.html`).
- Commit principal: `24f29ed` (26 archivos, 13172 inserciones, 5572 eliminaciones segun `git show --stat`) + commit de seguimiento `b1a3e75` (TODOS.md).

15. Mapeo del Ciclo de Vida por Especialistas gstack (Think -> Reflect)

Mapeo honesto de la sesion documentada en la Seccion 12 contra las 7 fases del ciclo gstack. Se marca explicitamente que fase se uso, cual no se uso y por que.

Fase	Skill(s)	Uso en esta sesion
Think	<code>/office-hours</code>	No usado. Se ofrecio explicitamente al usuario al inicio del pipeline de arquitectura; el usuario determino que no aplicaba porque <code>/office-hours</code> esta disenado para validar ideas de producto nuevas (demanda, wedge de mercado), no para auditar un sistema ya construido en produccion. La fase Think se cubrio en su lugar dentro del Paso 0 (Scope Challenge) de <code>/plan-eng-review</code> .
Plan	<code>/plan-eng-review</code>	Usado al inicio del trabajo de esta sesion: diagnostico arquitectonico completo (flujo de datos, deuda tecnica, limites de escalabilidad) antes de tocar cualquier codigo.
Build	Core Agent + <code>/careful</code>	Usado: implementacion directa de las 5 intervenciones (SEC-P0, SEC-P1, CONC-P2, NS-01, FE-01). <code>/careful</code> se invoco especificamente antes del refactor masivo de desacoplamiento de <code>Index.html</code> (FE-01), para activar advertencias sobre comandos destructivos durante esa fase.
Review	<code>/review</code> + <code>/cso</code>	<code>/review</code> usado 3 veces: auditoria inicial de codigo, verificacion de la implementacion de LockService/parches de seguridad, y verificacion final de la extraccion CSS/JS. <code>/cso</code> (Chief Security Officer) no fue invocado en esta sesion — las revisiones de seguridad se hicieron via <code>/review</code> con enfoque de Staff Engineer, no via el modo CSO dedicado.
Test	<code>/qa</code> + <code>/setup-browser-cookies</code>	Ambos invocados. <code>/setup-browser-cookies</code> no logro completarse de forma confiable en este entorno Windows/Git Bash (el proceso servidor del cookie-picker no sobrevive entre invocaciones del Bash tool). Ante eso, <code>/qa</code> se completo en modalidad de QA manual guiado: el usuario navego el mismo con su sesion autenticada real y confirmo el checklist punto por punto, en vez de automatizacion completa por navegador headless.
Ship	<code>clasp push</code> + <code>git commit</code>	Usado: <code>clasp push</code> de 37 archivos a un entorno de pruebas confirmado manualmente por el usuario como NO-produccion, seguido de <code>git commit</code> (<code>24f29ed</code> , luego <code>b1a3e75</code>). Sin <code>git push</code> a remoto — el repositorio no tiene <code>origin</code> configurado.
Reflect	<code>/sync-gbrain</code>	Usado repetidamente a lo largo de la sesion (tras cada bloque de trabajo). La etapa de codigo (<code>gbrain sources add</code>) falla de forma consistente por un problema de la ruta del proyecto (espacios y tildes en el path de Windows); la etapa de memoria (<code>gbrain import</code>) funciona correctamente. Tambien se uso <code>/context-save</code> para dejar un checkpoint de continuidad entre sesiones.

16. Manual de Gobierno Multi-Agente

Este proyecto se desarrolla con dos agentes de IA en paralelo sobre el mismo repositorio y la misma rama de git (`fix/qa-saveTracking-batch-lock` , sin remoto configurado). Esta seccion formaliza las reglas de convivencia para que ninguna sesion futura — de cualquiera de los dos agentes — rompa el trabajo de la otra.

16.1 AGENTES ACTIVOS

Agente	Modelo	Alcance historico en este documento
GitHub Copilot	GPT-5.4	Seccion 5 (flujo de staging/promocion, fallback de configuracion, mejoras de filtros UI, correccion de modal, wrappers de <code>cache_backend.gs</code>). Creo este documento y la politica de <code>CLAUDE.md</code> .
Claude Code	Claude Sonnet 5 (orquestrado con gstack v1.60.1.0)	Seccion 12 (SEC-P0, SEC-P1, CONC-P2, NS-01, FE-01) y Secciones 13-16 de este documento.

16.2 REGLAS DE CONVIVENCIA

- Un solo documento de evidencia.** `DOCUMENTACION_TECNICA_VIVA.md` es la unica bitacora tecnica del proyecto. Ningun agente debe crear un documento paralelo.
- Nunca borrar, solo complementar.** Ninguna sesion debe eliminar entradas de otra sesion. Mejoras de formato (como la introduccion de la plantilla 9.1) se aplican hacia adelante y se documentan como tal, sin reescribir silenciosamente el historial.
- Identificadores de tarea sin colision.** Los prefijos usados hasta ahora son `SEC-*` (seguridad), `CONC-*` (conurrencia), `NS-*` (namespace) y `FE-*` (frontend). Un agente que introduzca un prefijo nuevo debe registrarlo aqui para evitar reusar un identificador ya emitido por el otro agente.
- `TODOs.md` es el backlog compartido.** Es un documento distinto y complementario a `DOCUMENTACION_TECNICA_VIVA.md`: `TODOs.md` es “que falta por hacer”, el otro es “que se hizo y por que”. Ambos agentes deben leerlo antes de proponer trabajo nuevo, para no duplicar items ya identificados por el otro.
- Rama unica, sin remoto.** Todo el trabajo de ambos agentes vive hoy en la rama local `fix/qa-saveTracking-batch-lock`. Si en el futuro se configura un remoto o se crean ramas separadas por agente, esta seccion debe actualizarse con la nueva estrategia de integracion.
- `CLAUDE.md` es la fuente de reglas operativas para Claude Code.** La seccion “Documentacion viva obligatoria” de ese archivo es vinculante para cualquier sesion de Claude Code futura. Si GitHub Copilot usa un mecanismo equivalente de instrucciones persistentes, debe mantenerse alineado con la misma politica.

16.3 VERIFICACION CRUZADA DE HALLAZGOS

Cuando un agente corrige o complementa un hallazgo de la otra sesion (como la Seccion 12.6, que corrigio el diagnostico de disponibilidad de `make-pdf` hecho originalmente por la sesion de Copilot), debe: (a) no borrar la afirmacion original, (b) verificar el hallazgo de forma independiente antes de corregirlo, y (c) dejar explicito que fue corregido, cuando y por que agente.

17. Guia de Exportacion e Integracion Continuada (actualizada 2026-08-05)

17.1 EXPORTACION DIRECTA A WORD (VIA GSTACK MAKE-PDF CLI)

```
"C:\Users\LEON6\.claude\skills\gstack\make-pdf\dist\pdf.exe" generate DOCUMENTACION_TECNICA_VIVA.md Documento_Tecnico_Aplicacion_Predios.docx --to docx
```

17.2 EXPORTACION A PDF CON PORTADA Y TABLA DE CONTENIDOS

```
"C:\Users\LEON6\.claude\skills\gstack\make-pdf\dist\pdf.exe" generate DOCUMENTACION_TECNICA_VIVA.md
Documento_Tecnico_Aplicacion_Predios.pdf --to pdf --cover --toc
```

17.3 ESTADO VERIFICADO DE ESTOS COMANDOS (2026-08-05, VERIFICADO DOS VECES EL MISMO DIA)

Ver Seccion 12.6 para el diagnostico completo, incluyendo la correccion. Resumen final: los comandos de 17.1 y 17.2 **se ejecutaron realmente sobre este documento y funcionaron**, generando

`Documento_Tecnico_Aplicacion_Predios.docx` (529KB) y `Documento_Tecnico_Aplicacion_Predios.pdf` (1023KB, con portada y tabla de contenidos) en la raiz del repositorio. El unico fallo observado fue en `pdf.exe setup` (el smoke test interno de la herramienta, no usado por estos comandos), que sigue reportando `Blocked: scheme "about:" is not allowed` al intentar lanzar Chromium — no afecta a `generate`.

17.4 RUTA ALTERNATIVA (PANDOC)

Pandoc no esta instalado en este equipo a la fecha de esta verificacion (`command -v pandoc` sin resultado). No es necesario para exportar, dado que `make-pdf` ya cubre ambos formatos (17.1-17.2). Si se prefiere de todas formas, el comando equivalente seria:

```
pandoc DOCUMENTACION_TECNICA_VIVA.md -o Documento_Tecnico_Aplicacion_Predios.docx --toc
```